23. Scheduling: cron and systemd timers

23. Scheduling: cron and systemd timers

Phase: Phase 7 — Services and boot
Estimated time: 1–2 days
Slug: `23-cron-timers`

Learning objectives

- Write crontab entries
- Prefer systemd timers for modern systems
- Handle environment differences in scheduled jobs
- Log scheduled task output

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Backups, log rotation, certificate renewal, report generation, and cleanup jobs all need to run on a schedule. Linux offers two mainstream tools: classic `cron`, which is simple and everywhere, and `systemd` timers, which add logging, dependencies, and catch-up after downtime. Many “the job never ran” incidents come from five small things: cron’s minimal environment, the wrong time zone, a missing newline, a relative path, or output that went nowhere.

Core concepts

A crontab line has five time fields then a command:

```
+---------- minute        (0-59)
| +-------- hour          (0-23)
| | +------ day of month  (1-31)
| | | +---- month         (1-12)
| | | | +-- day of week   (0-7, 0 and 7 are Sunday)
| | | | |
0 3 * * *  /usr/local/bin/backup.sh
```

So `0 3 * * *` means every day at 03:00.

Expression	Meaning
`*/5 * * * *`	Every 5 minutes
`0 * * * *`	Every hour on the hour
`0 3 * * *`	Daily at 03:00
`30 2 * * 1`	Mondays at 02:30
`0 0 1 * *`	First of each month at midnight
`@reboot`	Once at boot
`0 9-17 * * 1-5`	On the hour, 09-17, Monday to Friday

Cron versus systemd timers:

Aspect	cron	systemd timer
Setup	One line in crontab	Two units: `.service` + `.timer`
Logs	Mail or your redirect	`journalctl -u name`
Missed runs	Lost	`Persistent=true` catches up
Environment	Minimal	Defined by the unit
Schedule syntax	Five fields	`OnCalendar=*-*-* 03:00:00`

Where jobs live:

```
crontab -e            -> per-user  (/var/spool/cron/...)
/etc/crontab          -> system, has an extra USER column
/etc/cron.d/*         -> drop-in system files, USER column
/etc/cron.daily/ ...  -> scripts run by run-parts
systemctl list-timers -> systemd timers
```

Worked examples

Edit and list your crontab:

```
crontab -e
crontab -l
```

Sample output:

```
# m h dom mon dow command
0 3 * * * /usr/local/bin/backup.sh >>/var/log/backup.log 2>&1
*/15 * * * * /usr/bin/curl -fsS https://example.com/health >/dev/null
```

Set environment explicitly, since cron's PATH is tiny:

```
SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin
```

```
MAILTO=ops@example.com
0 3 * * * cd /srv/app && ./cleanup.sh
```

Prevent overlapping runs with `flock`:

```
*/5 * * * * flock -n /tmp/job.lock /usr/local/bin/job.sh
```

A systemd timer pair:

```
sudo tee /etc/systemd/system/backup.service >/dev/null <<'EOF'
[Unit]
Description=Nightly backup
[Service]
Type=oneshot
ExecStart=/usr/local/bin/backup.sh
EOF
sudo tee /etc/systemd/system/backup.timer >/dev/null <<'EOF'
[Unit]
Description=Run backup daily at 03:00
[Timer]
OnCalendar=*-*-* 03:00:00
Persistent=true
[Install]
WantedBy=timers.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
systemctl list-timers --no-pager | head -3
```

Sample output:

```
NEXT                         LEFT     LAST PASSED UNIT         ACTIVATES
Thu 2026-10-22 03:00:00 UTC 16h left n/a  n/a    backup.timer backup.service
```

Validate a calendar expression:

```
systemd-analyze calendar "*-*-* 03:00:00"
```

Sample output:

```
Normalized form: *-*-* 03:00:00
    Next elapse: Thu 2026-10-22 03:00:00 UTC
```

Helper from the exercise: `cron_daily_3am() { echo "0 3 * * *"; }`

Mini-lab

1. Run `crontab -l` to see whether you have jobs.
2. Add `* * * * * date >> /tmp/cron-test.log` and wait two minutes.
3. Check `tail /tmp/cron-test.log`, then remove the line.
4. Write the cron expression for 02:30 every Monday and weekdays at 9.
5. Add `PATH=` and `SHELL=` at the top of your crontab and explain why.

6. Create the `backup.service` and `backup.timer` pair; run `systemctl start backup.service` manually and read its log with `journalctl -u backup.service`.
7. Run `systemd-analyze calendar` with three different expressions.
8. Use `flock -n` to make a job skip when the previous run is still active.

Decision guide / failure modes

Symptom	Cause	Fix
Works by hand, not in cron	Minimal env, relative paths	Absolute paths, set `PATH`
Nothing happened, no log	Output discarded	Redirect `>>file 2>&1`
Job missing after edit	No trailing newline or syntax error	Use `crontab -e`, check syslog
`%` breaks command	`%` means newline in crontab	Escape as `\%`
Ran at wrong hour	Time zone or DST	`timedatectl`, use UTC
Overlapping runs	Job slower than interval	`flock`
Missed while off	cron does not catch up	systemd `Persistent=true`

Choose cron for simple user-level jobs and quick one-liners; choose timers for services that need logs, dependencies, and catch-up. Always test the command as the same user with `env -i` to mimic cron's bare environment.

Common mistakes

- Relative paths in cron
- Emailing root floods from cron noise
- Duplicate cron+timer doing same job

Exercises

Work in `exercises/23-cron-timers/`. Read that folder's `README.md` fully before coding.

```
cd exercises/23-cron-timers
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement cron_daily_3am in exercise.sh. It prints the five-field crontab schedule expression for running a job once every day at 03:00 in the morning, and nothing else. The five fields are minute, hour, day of month, month and day of week. Because the value

contains asterisks, quote the string so the shell does not expand them into filenames. After the test passes, check your answer with systemd-analyze calendar using the equivalent OnCalendar form, and write the expression for 02:30 every Monday as a second drill.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/23-cron-timers/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Cron five fields meaning?
2. Why absolute paths?
3. Timer vs cron advantage?

Further reading

- https://man7.org/linux/man-pages/man5/crontab.5.html